for a hypothesis (such as system reliability) as more evidence or information becomes available. Bayesian inference can be used to estimate probabilistic quantities using historic data even when some information is missing. Using Bayesian techniques has allowed for real-time solutions to probability estimation problems that are beyond human reasoning [Tos17].

Proactive failure prediction using machine learning was discussed briefly in Section 15.4.3. It would be nice to be able to predict system failures in subsequent sprints before you deliver the prototype being developed in the current sprint. Making use of predictive data analytics such as a regression model involving MTBF has been used to estimate where and what types of defects might occur in future prototypes [Bat18].

A *genetic algorithm* is a heuristic search method used in artificial intelligence and computing. It is used for finding near-optimal solutions to search problems based on the theory of natural selection and evolutionary biology. Genetic algorithms can be used to grow reliability models by discovering relationships in historic system data. These models have been used to identify software components that may fail in the future. Sometimes these models have been created based on metrics estimated from UML models before any codes has been written [Pad17]. This type of work is very important to developers interested in refactoring software products or reusing software components in other products.

17.7.3 Software Safety

Software safety is a software quality assurance activity that focuses on the identification and assessment of potential hazards that may affect software negatively and cause an entire system to fail. If hazards can be identified early in the software process, software design features can be specified that will either eliminate or control potential hazards.

A modeling and analysis process is conducted as part of software safety. Initially, hazards are identified and categorized by criticality and risk. For example, some of the hazards associated with a computer-based cruise control for an automobile might be: (1) causes uncontrolled acceleration that cannot be stopped, (2) does not respond to depression of brake pedal (by turning off), (3) does not engage when switch is activated, and (4) slowly loses or gains speed. Once these system-level hazards are identified, analysis techniques are used to assign severity and probability of occurrence.[6] To be effective, software must be analyzed in the context of the entire system. For example, a subtle user input error (people are system components) may be magnified by a software fault to produce control data that improperly positions a mechanical device. If and only if a set of external environmental conditions is met, the improper position of the mechanical device will cause a disastrous failure. Analysis techniques [Eri15] such as fault tree analysis, real-time logic, and Petri net models can be used to predict the chain of events that can cause hazards and the probability that each of the events will occur to create the chain.

6 This approach is similar to the risk analysis methods described in Chapter 26. The primary difference is the emphasis on technology issues rather than project-related topics.

Once hazards are identified and analyzed, safety-related requirements can be specified for the software. That is, the specification can contain a list of undesirable events and the desired system responses to these events. The role of software in managing undesirable events is then indicated.

Although software reliability and software safety are related to one another, it is important to understand the subtle difference between them. Software reliability uses statistical analysis to determine the likelihood that a software failure will occur. However, the occurrence of a failure does not necessarily result in a hazard or mishap. Software safety examines the ways in which failures result in conditions that can lead to a mishap. That is, failures are not considered in a vacuum but are evaluated in the context of an entire computer-based system and its environment.

A comprehensive discussion of software safety is beyond the scope of this book. If you have further interest in software safety and related system issues, see [Fir13], [Har12a], and [Lev12].

17.8 THE ISO 9000 QUALITY STANDARDS[7]

A *quality assurance system* may be defined as the organizational structure, responsibilities, procedures, processes, and resources for implementing quality management [ANS87]. Quality assurance systems are created to help organizations ensure their products and services satisfy customer expectations by meeting their specifications. These systems cover a wide variety of activities encompassing a product's entire life cycle including planning, controlling, measuring, testing, and reporting, and improving quality levels throughout the development and manufacturing process. ISO 9000 describes quality assurance elements in generic terms that can be applied to any business regardless of the products or services offered.

To become registered to one of the quality assurance system models contained in ISO 9000, a company's quality system and operations are scrutinized by third-party auditors for compliance to the standard and for effective operation. Upon successful registration, a company is issued a certificate from a registration body represented by the auditors. Semiannual surveillance audits ensure continued compliance to the standard.

The requirements delineated by ISO 9001:2015 address topics such as management responsibility, quality system, contract review, design control, document and data control, product identification and traceability, process control, inspection and testing, corrective and preventive action, control of quality records, internal quality audits, training, servicing, and statistical techniques. For a software organization to become registered to ISO 9001:2015, it must establish policies and procedures to address each of the requirements just noted (and others) and then be able to demonstrate that these policies and procedures are being followed. If you desire further information on ISO 9001:2015, see [ISO14].

7 This section, written by Michael Stovsky, has been adapted from *Fundamentals of ISO 9000*, a workbook developed for *Essential Software Engineering,* a video curriculum developed by R. S. Pressman & Associates, Inc. Reprinted with permission.

INFO



The ISO 9001:2015 Standard

The following outline defines the basic elements of the ISO 9001:2015 standard. Comprehensive information on the standard can be obtained from the International Organization for Standardization (**www.iso.ch**) and other Internet sources (e.g., **www.praxiom.com**).

Establish the elements of a quality management system.
- Develop, implement, and improve the system.
- Define a policy that emphasizes the importance of the system.

Document the quality system.
- Describe the process.
- Produce an operational manual.
- Develop methods for controlling (updating) documents.
- Establish methods for record keeping.

Support quality control and assurance.
- Promote the importance of quality among all stakeholders.
- Focus on customer satisfaction.
- Define a quality plan that addresses objectives, responsibilities, and authority.
- Define communication mechanisms among stakeholders.

Establish review mechanisms for the quality management system.
- Identify review methods and feedback mechanisms.
- Define follow-up procedures.

Identify quality resources including personnel, training, and infrastructure elements.

Establish control mechanisms.
- For planning.
- For customer requirements.
- For technical activities (e.g., analysis, design, testing).
- For project monitoring and management.

Define methods for remediation.
- Assess quality data and metrics.
- Define approach for continuous process and quality improvement.

17.9 THE SQA PLAN

The *SQA plan* provides a road map for instituting software quality assurance. Developed by the SQA group (or by the software team if an SQA group does not exist), the plan serves as a template for SQA activities that are instituted for each software project.

A standard for SQA plans has been published by the IEEE [IEE17]. The standard recommends a structure that identifies: (1) the purpose and scope of the plan, (2) a description of all software engineering work products (e.g., models, documents, source code) that fall within the purview of SQA, (3) all applicable standards and practices that are applied during the software process, (4) SQA actions and tasks (including reviews and audits) and their placement throughout the software process, (5) the tools and methods that support SQA actions and tasks, (6) software configuration management (Chapter 22) procedures, (7) methods for assembling, safeguarding, and maintaining all SQA-related records, and (8) organizational roles and responsibilities relative to product quality.

17.10 SUMMARY

Software quality assurance is a software engineering umbrella activity that is applied at each step in the software process. SQA encompasses procedures for the effective application of methods and tools, oversight of quality control activities such as technical reviews and software testing, procedures for change management, procedures for assuring compliance to standards, and measurement and reporting mechanisms.

To properly conduct software quality assurance, data about the software engineering process should be collected, evaluated, and disseminated. Statistical SQA helps to improve the quality of the product and the software process itself. Software reliability models extend measurements, enabling collected defect data to be extrapolated into projected failure rates and reliability predictions.

In summary, you should note the words of Dunn and Ullman [Dun82]: "Software quality assurance is the mapping of the managerial precepts and design disciplines of quality assurance onto the applicable managerial and technological space of software engineering." The ability to ensure quality is the measure of a mature engineering discipline. When the mapping is successfully accomplished, mature software engineering is the result.

PROBLEMS AND POINTS TO PONDER

17.1. Some people say that "variation control is the heart of quality control." Because every program that is created is different from every other program, what are the variations that we look for and how do we control them?

17.2. Is it possible to assess the quality of software if the customer keeps changing what it is supposed to do?

17.3. Quality and reliability are related concepts but are fundamentally different in a number of ways. Discuss the differences.

17.4. Can a program be correct and still not be reliable? Explain.

17.5. Can a program be correct and still not exhibit good quality? Explain.

17.6. Why is there often tension between a software engineering group and an independent software quality assurance group? Is this healthy?

17.7. You have been given the responsibility for improving the quality of software across your organization. What is the first thing that you should do? What's next?

17.8. Besides counting errors and defects, are there other countable characteristics of software that imply quality? What are they and can they be measured directly?

17.9. The MTBF concept for reliability is open to criticism. Explain why.

17.10. Consider two safety-critical systems that are controlled by computer. List at least three hazards for each that can be directly linked to software failures.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

18 Software Security Engineering

Contributed by: Nancy Mead *Carnegie Mellon University Software Engineering Institute*

Stop and take a look around. Where do you see software being deployed? Sure, it's in your laptop, tablet, and cell phone. What about your appliances—refrigerator, dishwasher, and so forth? How about your car? Financial transactions—ATM, online banking, financial software, tax software? Your supplier of electricity? Definitely using software. Do you have any wearable devices on? A fit-bit? Maybe you have medical devices, like a pacemaker. The bottom line is: Software is all around us, on us, and sometimes in us. Every software product has the potential to be hacked, sometimes with dire consequences. This is the reason that we, as software engineers, need to be concerned about software security.

Key Concepts

attack patterns 363
attack surface 366
maturity models 370
measurement 368
misuse and abuse cases 363
requirements engineering 360
secure coding 367
secure development life-cycle activities 359
security life-cycle models 357
security process improvement 370
security risk analysis 364
software security engineering, importance of 357
threat modeling, prioritization, and mitigation 365

Quick Look

What is it? Software security engineering encompasses a set of techniques that can improve the security of software *while* it is under development.

Who does it? Although software engineers do not need to become security experts, they need to collaborate with security experts. The security experts may be members of the software team, on a separate specialized team, or they may be outside consultants.

Why is it important? The media continually reports instances of hacking—whether by gangsters, corporate competitors, hostile nations, or any other bad actor. The consequences for critical infrastructure, financial institutions, health care, and all aspects of modern life are significant.

What are the steps? There are a number of steps that can be taken to ensure that software is secure. We will discuss some of them here and provide pointers to resources for further exploration.

What is the work product? As you will see, there are many work products that are developed in the process of secure software engineering. The ultimate work product, or course, is the software that you have developed using secure software engineering practices.

How do I ensure that I've done it right? Everything that we will discuss as a method to improve software security, whether at the organizational or project level, can and should be reviewed by the interested parties. In addition, secure development processes can be improved, if they are found to be lacking.

18.1 Why Software Security Engineering Is Important

Software security is about more than just securing operational software with firewalls, strong passwords, and encryption. It's also about developing software in such a way that it is more secure from the outset. Techniques are now available that will help us develop software that is significantly more secure than it would be otherwise.

In this chapter, we're going to look as some of the models and techniques that can help us achieve better software security. We'll start by looking at security process models. Then we'll examine specific process activities, including such activities as requirements engineering, misuse or abuse cases, security risk analysis, threat modeling, attack surface, secure coding, and measurement. We'll also consider security process improvement models. Finally, we'll summarize and provide you with a list of references so that you can dive into any of these topics in more depth.

Software security engineering research is a very active area. In this book we're only providing an overview of methods and tools to support actual practice. There are many books (for example, [Mea16], [Shu13], and [Hel18]) and other resources devoted exclusively to software security engineering, and we will point you to some of them.

18.2 Security Life-Cycle Models

The Microsoft Security Development Lifecycle (SDL) [Mea16] [Mic18] is an industry-leading software security process. A Microsoft-wide initiative and a mandatory policy since 2004, the SDL enabled Microsoft to embed security and privacy in its software and culture. The SDL introduces security and privacy early and throughout all phases of the development process and is without question the most widely known and used security development life-cycle model.

Microsoft defined a collection of principles it calls *Secure by Design, Secure by Default, Secure in Deployment, and Communications* (SD3+C) to help determine where security efforts are needed. These are as follows [Mic10]:

Secure by Design

Secure architecture, design, and structure. Developers consider security issues part of the basic architectural design of software development. They review detailed designs for possible security issues, and they design and develop mitigations for all threats.

Threat modeling and mitigation. Threat models are created, and threat mitigations are present in all design and functional specifications.

Elimination of vulnerabilities. No known security vulnerabilities that would present a significant risk to the anticipated use of the software remain in the code after review. This review includes the use of analysis and testing tools to eliminate classes of vulnerabilities.

Improvements in security. Less secure legacy protocols and code are deprecated, and, where possible, users are provided with secure alternatives that are consistent with industry standards.

Secure by Default

Least privilege. All components run with the fewest possible permissions.

Defense in depth. Components do not rely on a single threat mitigation solution that leaves users exposed if it fails.